

Практическая работа: "Система управления киберспортивным турниром"

Цель работы

Разработать консольное приложение для управления киберспортивным турниром, демонстрирующее применение продвинутых концепций объектно-ориентированного программирования в Java.

Описание системы

Система должна моделировать киберспортивный турнир со следующими возможностями:

1. **Управление участниками:** регистрация команд и игроков
2. **Проведение матчей:** создание и обработка результатов матчей
3. **Статистика:** подсчет рейтингов и статистики игроков
4. **Валидация данных:** проверка корректности входных данных

Основные сущности системы

- **Участники:** команды и отдельные игроки
- **Матчи:** игры между командами с результатами
- **Турнир:** контейнер для участников и матчей
- **Статистика:** данные о результативности игроков

Техническое задание

Часть 1: Иерархия классов и интерфейсы

1.1 Базовый абстрактный класс Participant

Создайте абстрактный класс `Participant` с полями:

- `String name` - название участника
- `int rating` - рейтинг участника
- `LocalDateTime registrationDate` - дата регистрации

Заготовка кода:

```

public abstract class Participant {
    protected String name;
    protected int rating;
    protected LocalDateTime registrationDate;

    // TODO: Добавить конструкторы с перегрузкой
    // TODO: Реализовать equals(), hashCode(), toString()
    // TODO: Добавить абстрактный метод getType()
}

```

1.2 Класс Player

Создайте класс `Player`, наследующий от `Participant`:

- `String nickname` - игровой псевдоним
- `PlayerRole role` - игровая роль (enum)
- `int experience` - опыт в месяцах

1.3 Класс Team

Создайте класс `Team`, наследующий от `Participant`:

- `List<Player> players` - состав команды
- `Player captain` - капитан команды
- `String region` - регион команды

Требования:

- Максимум 5 игроков в команде
- Капитан должен быть из состава команды
- Перегрузка конструкторов для создания команды разными способами

1.4 Enum и интерфейсы

Создайте:

```

public enum PlayerRole {
    ENTRY_FRAGGER("Открывающий"),
    SUPPORT("Поддержка"),
    AWP("Снайпер"),
    IN_GAME_LEADER("Лидер"),
    RIFLER("Стрелок");

    // TODO: Добавить поля и методы
}

public enum GameDiscipline {

```

```

    CS2, DOTA2, LOL, VALORANT
}

public enum MatchStatus {
    SCHEDULED, IN_PROGRESS, FINISHED, CANCELLED
}

```

Создайте интерфейсы для ролей:

```

public interface Captain {
    void makeStrategyDecision(String strategy);
    List<Player> selectStartingLineup();
}

public interface Coach {
    void giveAdvice(Team team);
    void analyzeOpponent(Team opponent);
}

```

Часть 2: Records и Generics

2.1 Records для данных

Создайте записи:

```

public record MatchResult(
    String teamAName,
    String teamBName,
    int scoreA,
    int scoreB,
    LocalDateTime matchTime,
    GameDiscipline discipline
) {
    // TODO: Добавить методы для определения победителя
    // TODO: Валидация в конструкторе
}

public record PlayerStats(
    String playerNickname,
    int kills,
    int deaths,
    int assists,
    double rating
) {
    // TODO: Добавить вычисляемые методы (KDR, KAD)
}

```

2.2 Generic-класс Tournament

Создайте параметризованный класс:

```
public class Tournament<T extends Participant> {
    private String name;
    private GameDiscipline discipline;
    private List<T> participants;
    private List<MatchResult> matches;

    // TODO: Реализовать методы добавления участников
    // TODO: Создать метод для поиска участников по критериям
    // TODO: Добавить ограничения на типы участников
}
```

Часть 3: Функциональные интерфейсы

3.1 Использование встроенных функциональных интерфейсов

Реализуйте методы в Tournament:

```
// Фильтрация команд по рейтингу
public List<T> filterParticipants(Predicate<T> condition) {
    // TODO: Использовать Predicate для фильтрации
}

// Преобразование участников в строковое представление
public List<String> getParticipantNames(Function<T, String> nameExtractor) {
    // TODO: Использовать Function для преобразования
}

// Обновление рейтингов после матчей
public void updateRatings(Consumer<T> ratingUpdater) {
    // TODO: Использовать Consumer для обновления
}
```

3.2 Собственные функциональные интерфейсы

Создайте интерфейсы:

```
@FunctionalInterface
public interface MatchValidator {
    boolean isValidMatch(Team teamA, Team teamB);
}

@FunctionalInterface
public interface RatingCalculator {
    int calculateNewRating(int oldRating, MatchResult result, boolean isWinner);
}
```

Часть 4: Регулярные выражения

4.1 Валидация данных

Создайте класс `ValidationUtils` :

```
public class ValidationUtils {
    // Никнейм: латиница, цифры, _, -, длина 3-16 символов
    private static final String NICKNAME_PATTERN = "..."; // TODO

    // Название команды: любые символы, длина 2-30
    private static final String TEAM_NAME_PATTERN = "..."; // TODO

    public static boolean isValidNickname(String nickname) {
        // TODO: Использовать Pattern.matches()
    }

    public static boolean isValidTeamName(String teamName) {
        // TODO: Реализовать валидацию
    }
}
```

4.2 Парсинг результатов матчей

Создайте метод для парсинга строки результата:

```
// Формат: "TeamA 16:14 TeamB"
public static MatchResult parseMatchResult(String resultString) {
    String pattern = "..."; // TODO: создать паттерн
    // TODO: Использовать Pattern и Matcher
    // TODO: Извлечь названия команд и счет
}
```

Пример работы программы

=== Система управления турниром ===

1. Зарегистрировать команду
2. Добавить игрока в команду
3. Создать матч
4. Показать статистику
5. Выход

Выберите действие: 1

Введите название команды: Natus Vincere

Введите регион: Europe

Команда успешно зарегистрирована!

Выберите действие: 2

Введите никнейм игрока: simple

Введите роль (ENTRY_FRAGGER/SUPPORT/AWP/IN_GAME_LEADER/RIFLER): AWP

Выберите команду: Natus Vincere

Игрок добавлен в команду!

Примеры для понимания основных концепций

1. Функциональные интерфейсы - примеры использования

```
// Predicate - проверка условий
Predicate<Book> isAvailable = book -> !book.isBorrowed();
Predicate<Book> isNewBook = book -> book.getPublicationYear() >= 2020;
Predicate<Reader> hasActiveSubscription = reader -> reader.getSubscriptionEnd().isAfter(LocalDate.now());

// Function - преобразование данных
Function<Book, String> bookToDescription = book ->
    book.getTitle() + " - " + book.getAuthor();
Function<Reader, Integer> readerToAge = reader ->
    Period.between(reader.getBirthDate(), LocalDate.now()).getYears();

// Consumer - выполнение действий
Consumer<Book> markAsReturned = book -> {
    book.setBorrowed(false);
    book.setReturnDate(LocalDate.now());
};
Consumer<Reader> sendNotification = reader ->
    System.out.println("Уведомление отправлено читателю: " + reader.getFullName());

// Supplier - создание объектов
Supplier<List<Book>> createBookList = () -> new ArrayList<>();
Supplier<String> generateTicketNumber = () -> "T" + System.currentTimeMillis();
```

2. Records - примеры структур данных

```
// Информация о выдаче книги
public record Loan(
    String bookIsbn,
    String readerPassport,
    LocalDate loanDate,
    LocalDate dueDate
) {
    // Валидация в компактном конструкторе
    public Loan {
        if (dueDate.isBefore(loanDate)) {
            throw new IllegalArgumentException("Дата возврата не может быть раньше даты выдачи");
        }
    }
}
```

```

// Дополнительные методы
public boolean isOverdue() {
    return LocalDate.now().isAfter(dueDate);
}

public long daysOverdue() {
    return isOverdue() ?
        ChronoUnit.DAYS.between(dueDate, LocalDate.now()) : 0;
}
}

// Статистика библиотеки
public record LibraryStats(
    int totalBooks,
    int borrowedBooks,
    int availableBooks,
    int totalReaders
) {
    public double borrowingRate() {
        return totalBooks > 0 ? (double) borrowedBooks / totalBooks : 0.0;
    }
}

```

3. Generics - примеры параметризованных классов

```

// Общий класс для каталогов
public class Catalog<T> {
    private Map<String, T> items = new HashMap<>();

    public void addItem(String id, T item) {
        items.put(id, item);
    }

    public Optional<T> findItem(String id) {
        return Optional.ofNullable(items.get(id));
    }

    // Метод с ограничением типа
    public <U extends Comparable<U>> List<T> findItemsByComparableField(
        Function<T, U> fieldExtractor, U value) {

        return items.values().stream()
            .filter(item -> fieldExtractor.apply(item).compareTo(value) == 0)
            .collect(Collectors.toList());
    }
}

// Использование:

```

```
// Catalog<Book> bookCatalog = new Catalog<>();
// Catalog<Reader> readerCatalog = new Catalog<>();
```

4. Регулярные выражения - примеры паттернов

```
public class LibraryValidator {
    // ISBN формат: 978-5-389-12345-6
    private static final String ISBN_PATTERN =
        "^\\d{3}-\\d{1}-\\d{3}-\\d{5}-\\d{1}$";

    // Читательский билет: L + 6 цифр
    private static final String LIBRARY_CARD_PATTERN =
        "^L\\d{6}$";

    // Email библиотекаря
    private static final String LIBRARIAN_EMAIL_PATTERN =
        "^([a-zA-Z0-9._%+-]+@library\\. (ru|com|org))$";

    public static boolean isValidISBN(String isbn) {
        return isbn != null && isbn.matches(ISBN_PATTERN);
    }

    // Поиск всех упоминаний книг в тексте (заглавные буквы в кавычках)
    public static List<String> findBookTitles(String text) {
        Pattern pattern = Pattern.compile("\"([A-ZА-Я][^\" ]+)\"");
        Matcher matcher = pattern.matcher(text);
        List<String> titles = new ArrayList<>();

        while (matcher.find()) {
            titles.add(matcher.group(1));
        }
        return titles;
    }
}
```

5. Enum с методами

```
public enum BookGenre {
    FICTION("Художественная литература"),
    SCIENCE("Научная литература"),
    HISTORY("История"),
    BIOGRAPHY("Биографии"),
    CHILDREN("Детская литература");

    private final String russianName;

    BookGenre(String russianName) {
        this.russianName = russianName;
    }
}
```

```
public String getRussianName() {  
    return russianName;  
}  
  
// Метод для получения популярности жанра  
public int getPopularityScore() {  
    return switch(this) {  
        case FICTION -> 5;  
        case CHILDREN -> 4;  
        case HISTORY -> 3;  
        case BIOGRAPHY -> 2;  
        case SCIENCE -> 1;  
    };  
}  
}
```

Источники, которыми можно пользоваться

ГигаЧат не поможет (._.)